TRAVAIL ACADÉMIQUE

COMPRESSION ET QUANTIFICATION DE MODÈLES DE MACHINE LEARNING

Réduction de la taille des modèles pour les systèmes embarqués

INFORMATION	DÉTAIL
Étudiante	KATUKU LENGA Angélique
Formation	Master 1 — Intelligence Artificielle
Encadreur	Professeur Masakuna
Domaine	Apprentissage automatique et systèmes embarqués
Année académique	À compléter

Rapport consacré à l'étude de la réduction de taille des modèles d'apprentissage automatique par quantification, avec une expérimentation de classification d'images.

Document de travail — version à relire et à valider

Résumé

Les modèles de Machine Learning sont utilisés dans de nombreux domaines, mais leur déploiement sur des appareils embarqués peut être limité par la mémoire, le stockage, l'énergie et les capacités de calcul disponibles.

Ce travail étudie la quantification comme technique de réduction de la taille d'un modèle. L'expérience compare un modèle au format ONNX et une version quantifiée INT8, évalués pour la classification de chiffres manuscrits du jeu de données MNIST.

Les mesures obtenues dans l'environnement expérimental indiquent une taille de 428,07 Ko pour le modèle ONNX original et de 110,96 Ko pour le modèle quantifié. La réduction calculée est de 74,08 %, pour un facteur de compression d'environ 3,86.

La précision mesurée passe de 96,38 % à 96,34 %, soit une différence de -0,04 point de pourcentage. Ces résultats sont encourageants pour la réduction du stockage, mais ils ne suffisent pas à démontrer un gain de vitesse ou d'énergie sur un appareil embarqué réel.

Mots-clés : Machine Learning, quantification, INT8, ONNX, MNIST, compression, systèmes embarqués.

Abstract

Machine learning models are increasingly deployed in applications with limited computing and storage resources. Embedded systems often require smaller models while maintaining acceptable prediction quality.

This report investigates INT8 quantization as a model-size reduction technique. An original ONNX model is compared with a quantized ONNX version on a handwritten-digit classification task using the MNIST test dataset.

The recorded measurements indicate a model size reduction from 428.07 KB to 110.96 KB, corresponding to a reduction of 74.08% and a compression factor of approximately 3.86.

Measured classification accuracy changes from 96.38% to 96.34%, a difference of -0.04 percentage points. The results suggest that substantial storage reduction can be achieved with a small observed accuracy difference. Runtime, energy consumption and deployment on physical embedded hardware remain to be evaluated.

Keywords: Machine Learning, quantization, INT8, ONNX, MNIST, compression, embedded systems.

Sommaire

• 1. Introduction générale

• 2. Problématique et objectifs

• 3. Fondements du Machine Learning

• 4. Contraintes des systèmes embarqués

• 5. Compression et quantification

• 6. Présentation du jeu de données MNIST

• 7. Environnement et outils

• 8. Méthodologie expérimentale

• 9. Mesure de la taille des modèles

• 10. Évaluation de la précision

• 11. Présentation des résultats

• 12. Analyse et discussion

• 13. API et tests logiciels

• 14. Docker et intégration continue

• 15. Limites de l'étude

• 16. Perspectives

• 17. Conclusion générale

• 18. Bibliographie

• 19. Annexes techniques

Remarque : la pagination définitive dépend de la mise en page et peut varier si le document est modifié.

1. Introduction générale

L'intelligence artificielle et l'apprentissage automatique occupent une place croissante dans les systèmes informatiques modernes. Des modèles sont employés pour la reconnaissance d'images, le traitement du langage, la détection d'anomalies et l'aide à la décision.

Les modèles performants peuvent toutefois présenter des coûts de stockage et d'exécution qui compliquent leur utilisation sur des plateformes de faible puissance. Les appareils embarqués disposent souvent de mémoire limitée et fonctionnent dans des conditions énergétiques ou thermiques contraignantes.

La compression des modèles cherche à diminuer les ressources nécessaires au stockage et à l'exécution. La quantification constitue une approche importante : elle remplace certaines représentations numériques par des formats de précision plus faible, par exemple des entiers sur huit bits.

Ce rapport examine cette démarche à travers une expérience de classification de chiffres manuscrits. Le modèle original est comparé à sa version quantifiée selon deux indicateurs principaux : la taille du fichier et la précision de classification.

L'étude distingue les mesures effectivement obtenues des performances qui nécessiteraient des expériences supplémentaires, notamment le temps d'inférence et la consommation énergétique.

2. Problématique et objectifs

Problématique. Comment réduire la taille d'un modèle de Machine Learning destiné à un environnement contraint tout en préservant une qualité de prédiction satisfaisante ?

La réduction de taille peut faciliter le stockage et le transfert du modèle. Toutefois, une représentation numérique moins précise peut modifier les valeurs internes du réseau et entraîner une variation des prédictions.

Objectif général. Étudier l'effet de la quantification INT8 sur la taille et la précision d'un modèle de classification.

Objectifs spécifiques :

• Comparer la taille du modèle ONNX original et de sa version INT8.

• Calculer le pourcentage de réduction et le facteur de compression.

• Mesurer la précision des deux versions sur un jeu de test.

• Interpréter le compromis entre compacité et précision.

• Documenter les étapes et identifier les validations encore nécessaires.

Question de recherche : la quantification observée permet-elle de réduire sensiblement la taille du modèle sans diminution importante de la précision mesurée ?

L'hypothèse de travail est qu'une quantification correctement appliquée peut réduire la taille du modèle avec une variation limitée des performances de classification.

3. Fondements du Machine Learning

Le Machine Learning est un domaine de l'intelligence artificielle dans lequel un système apprend des régularités à partir de données. Dans l'apprentissage supervisé, les exemples sont accompagnés d'étiquettes servant de références pendant l'entraînement.

Un modèle de classification reçoit une entrée et produit une sortie correspondant à une classe. Dans la reconnaissance des chiffres, les classes possibles sont les valeurs de zéro à neuf.

Un réseau de neurones artificiels est constitué de couches de calcul. Chaque couche transforme les représentations reçues en appliquant des opérations numériques et, généralement, des fonctions d'activation.

L'entraînement ajuste les paramètres du modèle afin de réduire une fonction de perte. L'évaluation utilise ensuite des données séparées afin d'estimer la capacité de généralisation.

La précision est la proportion d'exemples correctement classés. Elle est simple à interpréter, mais ne décrit pas à elle seule tous les comportements d'un modèle. D'autres métriques peuvent être nécessaires pour des données déséquilibrées ou des erreurs dont les coûts diffèrent.

Dans cette étude, la précision sert à comparer le comportement du modèle original et celui du modèle quantifié sur le même jeu de test.

4. Contraintes des systèmes embarqués

Un système embarqué est un dispositif informatique intégré à un équipement plus large. Il peut s'agir d'un capteur, d'un appareil domestique, d'un équipement industriel ou d'un système robotique.

Ces plateformes peuvent présenter des limites de mémoire vive, de stockage, de puissance de calcul et d'alimentation électrique. Elles peuvent également être soumises à des contraintes de délai et de température.

La taille d'un modèle influence la quantité de stockage nécessaire et peut avoir un effet sur le transfert du modèle. Elle ne détermine cependant pas, à elle seule, la mémoire d'exécution ni la latence.

La latence dépend notamment du processeur, du moteur d'inférence, des opérations du réseau et de la taille des entrées. La consommation énergétique doit être mesurée sur la plateforme visée.

Une démarche de déploiement embarqué doit donc considérer plusieurs indicateurs : taille du fichier, mémoire utilisée, latence, débit de traitement, précision et consommation énergétique.

Le présent travail quantifie principalement la taille et la précision. Les autres dimensions restent des pistes d'évaluation ultérieure sur du matériel réel.

5. Compression et quantification des modèles

La compression des modèles regroupe plusieurs techniques visant à diminuer les ressources nécessaires au stockage ou à l'exécution. Ces techniques peuvent être utilisées séparément ou combinées.

Élagage. L'élagage supprime ou neutralise certains paramètres ou certaines connexions considérés comme peu utiles. Le gain réel dépend de la structure obtenue et du support logiciel ou matériel des représentations clairsemées.

Partage de poids. Plusieurs paramètres peuvent être représentés par des valeurs communes, réduisant la quantité d'information à stocker.

Distillation. Un modèle plus petit apprend à reproduire une partie du comportement d'un modèle enseignant plus complexe.

Quantification. La quantification représente les valeurs avec une précision numérique réduite. Elle peut concerner les poids, les activations ou les deux, selon la méthode.

Les méthodes ne présentent pas les mêmes compromis. Le choix dépend de l'architecture, du niveau de précision acceptable, des outils disponibles et du matériel cible.

Dans cette expérience, l'attention porte sur une version ONNX quantifiée en INT8 et sur sa comparaison avec le modèle ONNX original.

6. Principes de la quantification INT8

Le format INT8 utilise des entiers signés représentés sur huit bits. Il offre 256 valeurs distinctes, généralement comprises entre -128 et 127 pour une représentation signée.

Une quantification affine peut être décrite par une échelle et un point zéro. Une valeur réelle est approchée par une valeur entière quantifiée, puis interprétée au moyen de ces paramètres.

La précision réduite peut diminuer le volume de stockage des tenseurs et permettre l'utilisation d'opérations entières lorsque le moteur d'inférence les prend en charge.

La quantification dynamique détermine certaines informations de quantification au moment de l'exécution. D'autres approches utilisent une calibration préalable sur des données représentatives.

Il est important de distinguer le format numérique, la méthode de quantification et le format de fichier. Un fichier ONNX quantifié ne garantit pas que toutes les opérations sont exécutées en INT8 sur chaque plateforme.

La quantification peut introduire des erreurs d'arrondi. La précision doit donc être mesurée après transformation, et les résultats doivent être comparés sur les mêmes exemples et avec le même prétraitement.

7. Présentation du jeu de données MNIST

MNIST est un jeu de données de référence pour la reconnaissance de chiffres manuscrits. Les images représentent des chiffres de zéro à neuf en niveaux de gris.

Les images ont une résolution de 28 × 28 pixels. Chaque image est associée à une étiquette indiquant le chiffre représenté. Le jeu classique comporte 60 000 exemples d'entraînement et 10 000 exemples de test.

La simplicité relative de MNIST en fait un support pédagogique utile pour étudier les réseaux de neurones, les métriques de classification et les effets de transformations numériques.

Une évaluation correcte exige de reproduire le prétraitement attendu par le modèle : dimensions, type numérique, plage des valeurs et ordre des dimensions.

Il faut également s'assurer que les données de test ne servent pas à ajuster le modèle. Le jeu de test doit rester une mesure indépendante de la performance finale.

Dans l'expérience rapportée, la précision a été calculée sur les 10 000 exemples de test MNIST, avec les résultats enregistrés dans l'environnement Google Colab.

8. Environnement expérimental et outils

Python. Le langage Python sert à orchestrer les étapes de chargement des données, d'évaluation et d'enregistrement des résultats.

PyTorch. PyTorch est un cadre de calcul utilisé dans les projets de réseaux de neurones. Le fichier original du projet porte l'extension PTH.

ONNX. ONNX définit un format d'échange pour représenter des modèles d'apprentissage automatique. Il facilite le passage entre des outils de création et des environnements d'exécution.

ONNX Runtime. Ce moteur permet de charger et d'exécuter des modèles ONNX. Le fournisseur d'exécution utilisé peut influencer les performances observées.

torchvision et MNIST. Ces outils permettent de charger les données de chiffres manuscrits et de les transformer.

Google Colab. L'environnement notebook permet d'exécuter le code et de conserver les fichiers dans l'espace de travail de la session.

Les mesures présentées dans ce rapport sont celles consignées au cours de l'expérience. Les versions précises des bibliothèques et les caractéristiques du matériel devraient être ajoutées si elles sont disponibles.

9. Méthodologie expérimentale

L'expérience suit une comparaison entre deux artefacts : le modèle ONNX original et le modèle ONNX quantifié INT8. Les deux fichiers sont évalués sur le même jeu de test.

La première étape consiste à vérifier la présence des fichiers et à mesurer leur taille sur disque. La deuxième consiste à charger les deux modèles dans ONNX Runtime.

La troisième étape parcourt les exemples de test, effectue une prédiction avec chaque modèle et compare la classe prédite à l'étiquette attendue.

Le nombre de prédictions correctes est divisé par le nombre total d'exemples pour obtenir la précision en pourcentage.

Les résultats sont ensuite comparés. La réduction de taille est calculée à partir de la taille des deux fichiers ONNX. La différence de précision est exprimée en points de pourcentage.

Pour garantir la reproductibilité, il est recommandé d'enregistrer les versions des bibliothèques, la méthode exacte de quantification, le prétraitement et les paramètres du modèle.

Le protocole doit utiliser les mêmes données, les mêmes étiquettes et les mêmes conventions de prédiction pour les deux versions.

10. Mesure de la taille des modèles

Les tailles relevées sont de 428,07 Ko pour le modèle ONNX original et de 110,96 Ko pour le modèle ONNX quantifié INT8.

L'économie de stockage est la différence entre ces deux valeurs :

Économie = 428,07 – 110,96 = 317,11 Ko.

La réduction en pourcentage est calculée par la formule :

Réduction (%) = (Taille originale – Taille quantifiée) / Taille originale × 100.

En remplaçant les valeurs : (428,07 – 110,96) / 428,07 × 100, on obtient environ 74,08 %.

Le facteur de compression est le rapport entre la taille originale et la taille quantifiée : 428,07 / 110,96, soit environ 3,86.

Ces indicateurs décrivent la taille des fichiers observés. Ils ne mesurent pas directement la mémoire vive consommée pendant l'inférence ni le temps nécessaire à une prédiction.

Indicateur	Valeur
ONNX original	428,07 Ko
ONNX quantifié INT8	110,96 Ko
Économie de stockage	317,11 Ko
Réduction calculée	74,08 %
Facteur de compression	3,86×

11. Évaluation de la précision

La précision mesure la proportion d'exemples correctement classés parmi l'ensemble des exemples évalués. Elle est calculée par le rapport entre le nombre de prédictions correctes et le nombre total d'exemples, multiplié par 100.

La précision mesurée pour le modèle original est de 96,38 %. La précision mesurée pour le modèle quantifié INT8 est de 96,34 %.

La différence est obtenue en soustrayant la précision originale de la précision quantifiée : 96,34 – 96,38 = -0,04 point de pourcentage.

La différence observée est faible. Elle indique que, dans cette évaluation, la quantification n'a entraîné qu'une variation limitée de la précision globale.

Cette interprétation doit rester liée au protocole utilisé. Une seule métrique ne permet pas de décrire toutes les erreurs possibles, et des essais complémentaires peuvent être nécessaires pour confirmer la robustesse du résultat.

La comparaison est pertinente à condition que le prétraitement soit identique, que les sorties soient interprétées correctement et que les deux modèles soient évalués sur les mêmes exemples.

12. Présentation des résultats

Le tableau ci-dessous récapitule les mesures consignées pendant l'expérience.

Indicateur	ONNX original	ONNX quantifié INT8
Taille	428,07 Ko	110,96 Ko
Précision	96,38 %	96,34 %
Taux d'erreur	3,62 %	3,66 %

Lecture du tableau

Le modèle quantifié est nettement plus petit que le modèle original. La différence de taille est de 317,11 Ko. En parallèle, la précision diminue de 0,04 point de pourcentage.

Principaux constats

• La taille du fichier ONNX quantifié est environ 3,86 fois plus faible.

• La réduction de taille calculée est de 74,08 %.

• La précision observée demeure supérieure à 96 % pour les deux modèles.

• La vitesse d'inférence n'a pas été mesurée dans cette expérience.

• Le déploiement sur un dispositif embarqué réel n'a pas été validé.

13. Analyse et discussion

Les résultats mettent en évidence un compromis favorable entre taille de fichier et précision mesurée. La version quantifiée occupe beaucoup moins d'espace tout en conservant une précision proche de celle de la version originale.

La réduction de taille peut être intéressante pour le stockage local, la distribution d'un modèle et son transfert vers des appareils à capacité limitée. Cet avantage est directement étayé par les mesures de taille.

La faible différence de précision suggère que la quantification a préservé une grande partie du comportement du modèle sur le jeu de test utilisé. Il ne faut cependant pas généraliser ce résultat à d'autres modèles ou à d'autres données sans expérience.

Un gain de stockage n'implique pas automatiquement un gain de latence. Certaines plateformes accélèrent les opérations entières, tandis que d'autres peuvent exécuter certaines opérations quantifiées sans avantage particulier.

Une évaluation complète du déploiement embarqué devrait mesurer le temps d'inférence avec plusieurs répétitions, la mémoire utilisée et, si possible, la consommation énergétique.

Il serait aussi utile de vérifier les prédictions différentes entre les deux modèles, et de produire une matrice de confusion afin d'identifier les classes qui subissent une éventuelle dégradation.

14. API et tests logiciels

Une API permet d'exposer un modèle de prédiction à d'autres applications. FastAPI est un cadre Python qui permet de définir des routes HTTP et de valider les données reçues.

Dans un projet de classification d'images, une API peut recevoir les pixels d'une image, appliquer le prétraitement attendu, exécuter le modèle et renvoyer la classe prédite.

Les tests automatisés vérifient le comportement attendu de l'application. Ils peuvent notamment tester une requête valide, une entrée de dimension incorrecte et le format de la réponse.

La présence des fichiers app.py et test_app.py a été vérifiée dans l'environnement du projet. Des tests ont également été exécutés avec succès selon les étapes consignées.

Cette vérification ne constitue pas une preuve exhaustive de la qualité de l'API. Il reste nécessaire de vérifier que le modèle utilisé par l'API est bien chargé, que les entrées sont correctement prétraitées et que les tests couvrent les principaux cas d'usage.

L'intégration de l'API au modèle quantifié doit être explicitement validée avant de présenter le système comme entièrement opérationnel.

15. Docker et intégration continue

Docker permet de regrouper une application et ses dépendances dans une image afin de faciliter sa reproduction dans différents environnements. Un Dockerfile décrit les instructions de construction.

La présence d'un Dockerfile ne prouve pas que l'image se construit correctement. Une validation nécessite une construction effective, puis le lancement du conteneur et un test de l'application.

L'intégration continue automatise des tâches telles que la vérification syntaxique, l'installation des dépendances et l'exécution des tests.

Le fichier .gitlab-ci.yml définit des étapes de pipeline destinées à être exécutées par GitLab CI/CD. Sa création locale ne signifie pas que le pipeline a déjà été exécuté par un runner GitLab.

Pour valider la chaîne logicielle, il faut publier le dépôt, configurer l'environnement, vérifier les dépendances et examiner les journaux d'exécution du pipeline.

Dans le cadre de ce rapport, Docker et GitLab CI/CD sont considérés comme des éléments préparés, mais dont la validation complète reste à réaliser.

16. Limites de l'étude

Cette expérience présente plusieurs limites. Premièrement, les mesures disponibles concernent la taille des fichiers et la précision de classification. Elles ne comprennent pas de mesure du temps d'inférence ni de consommation énergétique.

Deuxièmement, les tests ont été réalisés dans un environnement logiciel et non sur une plateforme embarquée physique. Les performances peuvent varier en fonction du processeur, du moteur d'exécution et de la prise en charge des opérations quantifiées.

Troisièmement, la description détaillée des versions des outils, de la méthode exacte de quantification et des paramètres de calibration doit être conservée pour faciliter la reproduction.

Quatrièmement, la précision globale ne permet pas de comprendre toutes les différences de comportement. Une analyse des erreurs et une matrice de confusion apporteraient des informations utiles.

Enfin, la préparation des fichiers Docker et GitLab CI/CD ne remplace pas leur exécution effective. Leur fonctionnement doit être vérifié avant toute affirmation de déploiement réussi.

Ces limites n'annulent pas les résultats de taille et de précision mesurés, mais elles définissent la portée raisonnable des conclusions.

17. Perspectives

Une première perspective consiste à mesurer la latence du modèle original et du modèle quantifié. Les mesures devraient être répétées sur la même plateforme, avec une phase d'échauffement et un nombre suffisant d'exécutions.

Une deuxième perspective est de mesurer la mémoire utilisée au cours de l'inférence. La taille du fichier ne reflète pas directement la mémoire de travail requise par le moteur d'exécution.

Une troisième perspective est d'évaluer la consommation énergétique sur une plateforme représentative du déploiement envisagé.

Une comparaison de plusieurs stratégies serait également utile : quantification dynamique, quantification avec calibration et éventuellement quantification consciente de l'entraînement.

Les résultats pourraient être étendus à d'autres architectures ou à des jeux de données plus représentatifs de l'application finale.

Enfin, le projet pourrait être complété par une API testée de bout en bout, une image Docker construite avec succès et un pipeline GitLab exécuté sur un runner. Ces étapes renforceraient la reproductibilité et la qualité du livrable logiciel.

18. Conclusion générale

Ce travail avait pour objectif d'étudier la compression et la quantification des modèles de Machine Learning dans le contexte des systèmes embarqués.

L'expérience a comparé un modèle ONNX original à une version quantifiée INT8 sur une tâche de reconnaissance de chiffres manuscrits à partir du jeu de test MNIST.

Les tailles mesurées sont de 428,07 Ko pour le modèle original et de 110,96 Ko pour le modèle quantifié. La réduction calculée est de 74,08 %, soit une économie de stockage de 317,11 Ko et un facteur de compression d'environ 3,86.

La précision mesurée passe de 96,38 % à 96,34 %, ce qui correspond à une différence de -0,04 point de pourcentage. Dans le cadre de cette expérience, la quantification a donc permis une réduction importante de la taille avec une variation limitée de la précision.

Ces résultats sont encourageants pour les applications où le stockage est une contrainte. Toutefois, ils ne démontrent pas encore un gain de vitesse ou d'énergie sur du matériel embarqué.

Les travaux futurs devraient mesurer la latence, la mémoire et la consommation énergétique, puis valider le déploiement complet de l'API et de l'environnement logiciel.

En conclusion, la quantification INT8 apparaît comme une technique prometteuse, à condition que ses effets soient évalués dans le contexte matériel et applicatif visé.

19. Bibliographie

1. LeCun, Y., Cortes, C. et Burges, C. J. C. The MNIST Database of Handwritten Digits. Site de référence : yann.lecun.com/exdb/mnist/.

2. Jacob, B. et al. (2018). Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR).

3. Krishnamoorthi, R. (2018). Quantizing deep convolutional networks for efficient inference: A whitepaper. arXiv:1806.08342.

4. ONNX. Documentation officielle du format Open Neural Network Exchange. https://onnx.ai/.

5. ONNX Runtime. Documentation officielle, quantification des modèles. https://onnxruntime.ai/docs/performance/model-optimizations/quantization.html.

6. PyTorch. Documentation officielle relative à la quantification et à l'optimisation des modèles. https://pytorch.org/docs/.

7. FastAPI. Documentation officielle du cadre de développement d'API Python. https://fastapi.tiangolo.com/.

8. Docker. Documentation officielle concernant les images et les Dockerfiles. https://docs.docker.com/.

9. GitLab. Documentation officielle de GitLab CI/CD. https://docs.gitlab.com/ci/.

10. Rapport expérimental de l'étudiante : mesures enregistrées dans Google Colab le 9 octobre 2026. Tailles de fichiers et précisions rapportées dans ce document.

Les documentations officielles doivent être consultées dans leur version correspondant aux bibliothèques utilisées. Les références bibliographiques doivent être harmonisées selon les règles de l'établissement.

20. Annexe A — Formules utilisées

Réduction de taille.

Réduction (%) = ((Taille originale – Taille quantifiée) / Taille originale) × 100.

Application : ((428,07 – 110,96) / 428,07) × 100 ≈ 74,08 %.

Économie de stockage.

Économie = Taille originale – Taille quantifiée.

Application : 428,07 – 110,96 = 317,11 Ko.

Facteur de compression.

Facteur = Taille originale / Taille quantifiée.

Application : 428,07 / 110,96 ≈ 3,86.

Précision.

Précision (%) = (Nombre de prédictions correctes / Nombre total d'exemples) × 100.

Différence de précision.

Différence = Précision quantifiée – Précision originale.

Application : 96,34 – 96,38 = -0,04 point de pourcentage.

Les résultats doivent être calculés à partir des mesures précises enregistrées. Les valeurs affichées sont arrondies à deux décimales.

21. Annexe B — Organisation des livrables

Les fichiers de résultats préparés dans Google Colab sont organisés dans le dossier livrables.

resultats.csv. Tableau contenant les mesures de taille et de précision du modèle original et du modèle quantifié.

comparaison_taille.png. Graphique illustrant la différence de taille entre les deux fichiers ONNX.

rapport.md. Version textuelle du rapport expérimental.

Les fichiers techniques du projet comprennent également, selon leur présence dans l'environnement, l'application app.py, les tests test_app.py, le Dockerfile, le fichier .gitlab-ci.yml et les modèles enregistrés.

Il est recommandé de conserver une copie séparée du code source, des modèles et des résultats. Avant la remise, il convient de vérifier le contenu réel de l'archive ZIP et d'ouvrir les fichiers pour s'assurer qu'ils ne sont pas vides.

La présence d'un fichier dans un dossier ne garantit pas son fonctionnement. Les scripts doivent être exécutés dans un environnement correctement configuré pour valider l'application.

22. Annexe C — Liste de vérification avant dépôt

Point de contrôle	État
Nom et formation sur la page de garde	Présents
Résultats de taille documentés	Mesurés
Précision originale et quantifiée documentée	Mesurée
Calcul de réduction documenté	Calculé
Graphique comparatif	Créé
Rapport PDF	À relire
Versions des bibliothèques	À compléter si disponibles
Temps d'inférence sur appareil embarqué	Non mesuré
Consommation énergétique	Non mesurée
Construction effective de l'image Docker	À vérifier
Exécution effective du pipeline GitLab	À vérifier
Validation finale par l'encadreur	À effectuer

Cette liste doit être utilisée pour présenter honnêtement l'état du projet. Les éléments non réalisés ne doivent pas être déclarés comme terminés.